

ASS 19.3

2303A52346

Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:

- o Constructors

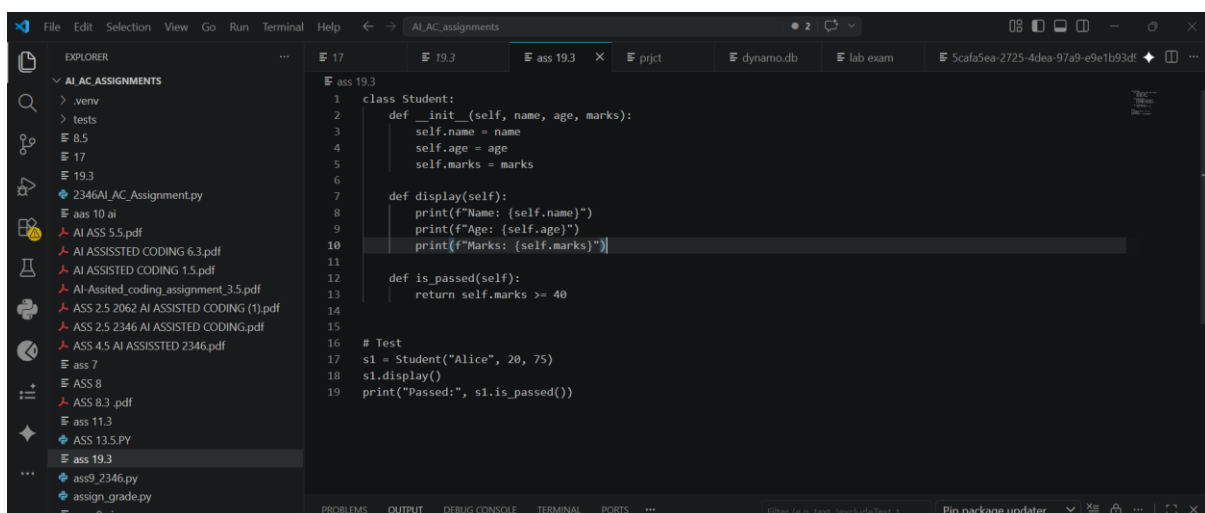
- o Data types

- o Access specifiers

- Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.



```
1 class Student:
2     def __init__(self, name, age, marks):
3         self.name = name
4         self.age = age
5         self.marks = marks
6
7     def display(self):
8         print(f"Name: {self.name}")
9         print(f"Age: {self.age}")
10        print(f"Marks: {self.marks}")
11
12    def is_passed(self):
13        return self.marks >= 40
14
15
16 # Test
17 s1 = Student("Alice", 20, 75)
18 s1.display()
19 print("Passed:", s1.is_passed())
```

Convert a Java method that checks whether a number is prime into

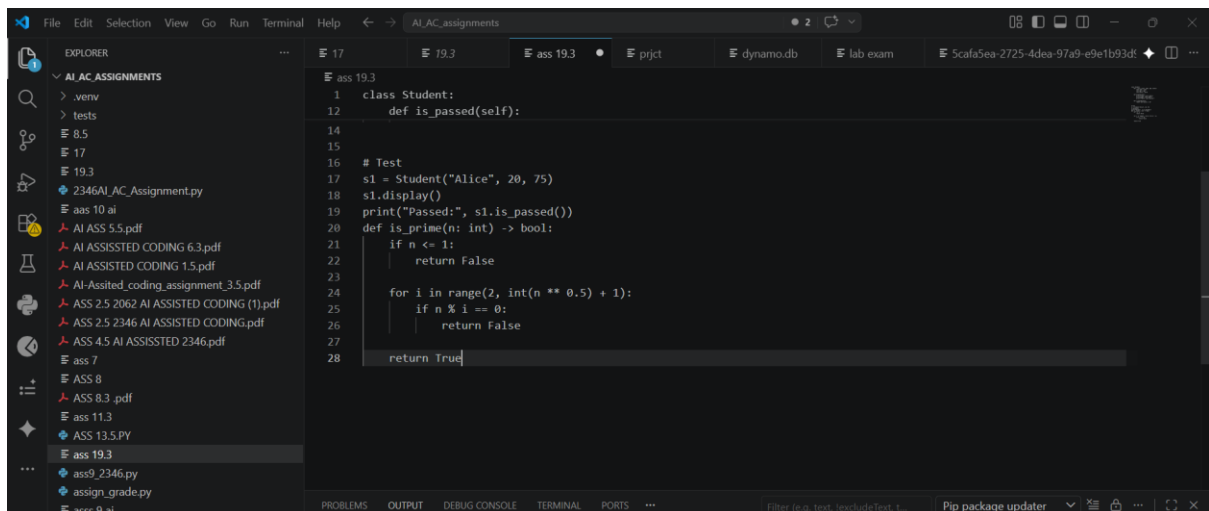
Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method



The screenshot shows a VS Code editor window with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'AI.AC_ASSIGNMENTS' with various files including '2346AI.AC_Assignment.py' and 'ass 19.3'. The code editor shows the following Python code:

```
1 class Student:
12     def is_passed(self):
14
15
16     # Test
17     s1 = Student("Alice", 20, 75)
18     s1.display()
19     print("Passed:", s1.is_passed())
20 def is_prime(n: int) -> bool:
21     if n <= 1:
22         return False
23
24     for i in range(2, int(n ** 0.5) + 1):
25         if n % i == 0:
26             return False
27
28     return True
```

Pseudocode to Python Implementation

Task:

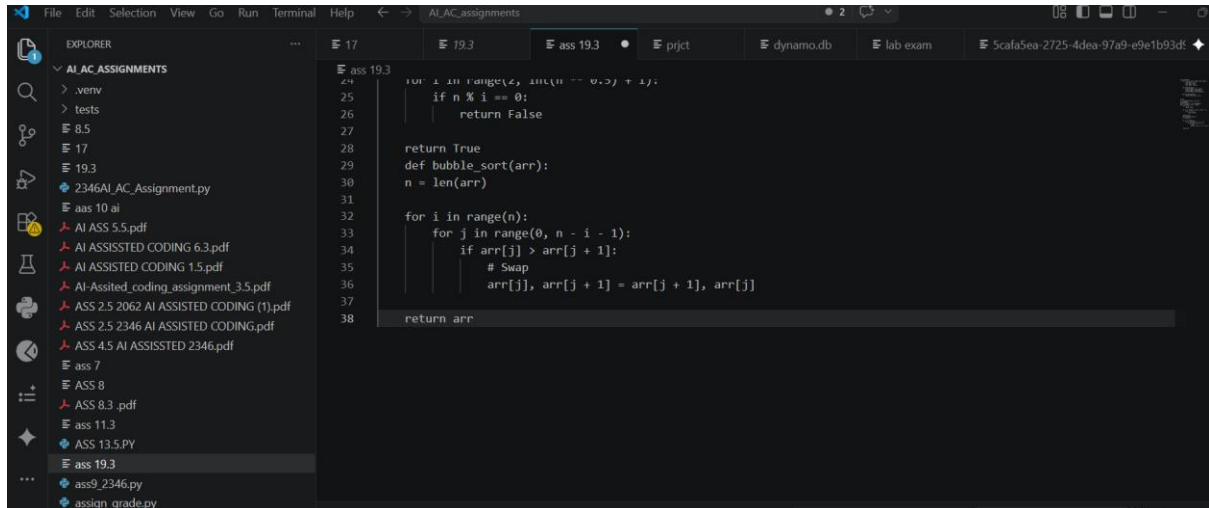
Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

A screenshot of a Visual Studio Code editor window. The Explorer sidebar on the left shows a project named 'AI AC ASSIGNMENTS' with various files including '.venv', 'tests', '8.5', '17', '19.3', '2346AI_AC_Assignment.py', 'aas 10 ai', 'AI ASS 5.5.pdf', 'AI ASSISTED CODING 6.3.pdf', 'AI ASSISTED CODING 1.5.pdf', 'AI-Assited_coding_assignment_3.5.pdf', 'ASS 2.5 2062 AI ASSISTED CODING (1).pdf', 'ASS 2.5 2346 AI ASSISTED CODING.pdf', 'ASS 4.5 AI ASSISTED 2346.pdf', 'ass 7', 'ASS 8', 'ASS 8.3 .pdf', 'ass 11.3', 'ASS 13.5.PY', 'ass 19.3', 'ass9_2346.py', and 'assign_grade.py'. The main editor area shows a Python file named 'ass 19.3' with the following code:

```
24 for i in range(1, len(arr) - 1):
25     if n % i == 0:
26         return False
27
28 return True
29 def bubble_sort(arr):
30     n = len(arr)
31
32     for i in range(n):
33         for j in range(0, n - i - 1):
34             if arr[j] > arr[j + 1]:
35                 # Swap
36                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
37
38 return arr
```

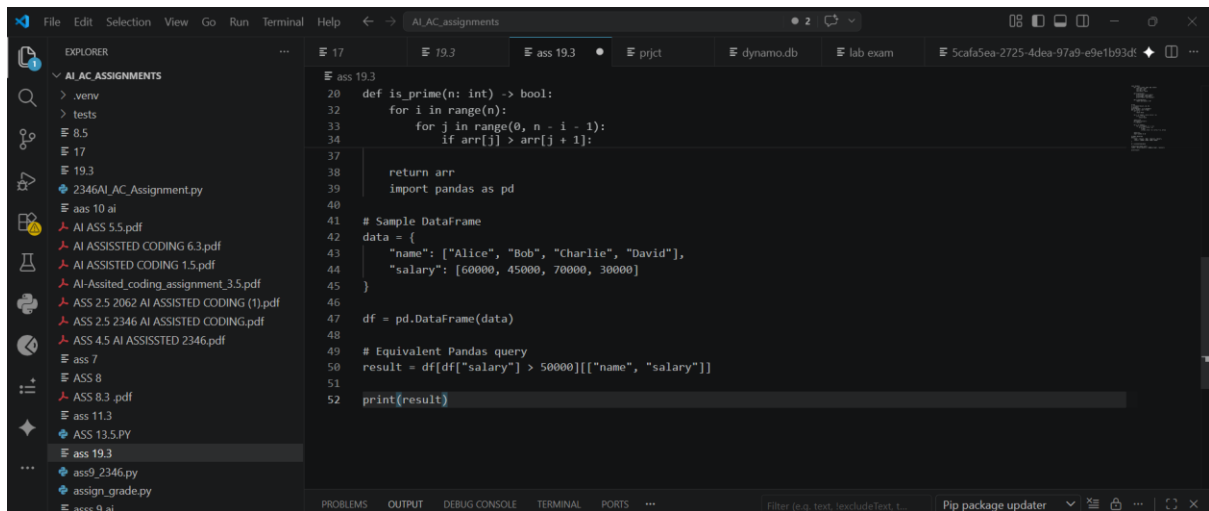
Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.



Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

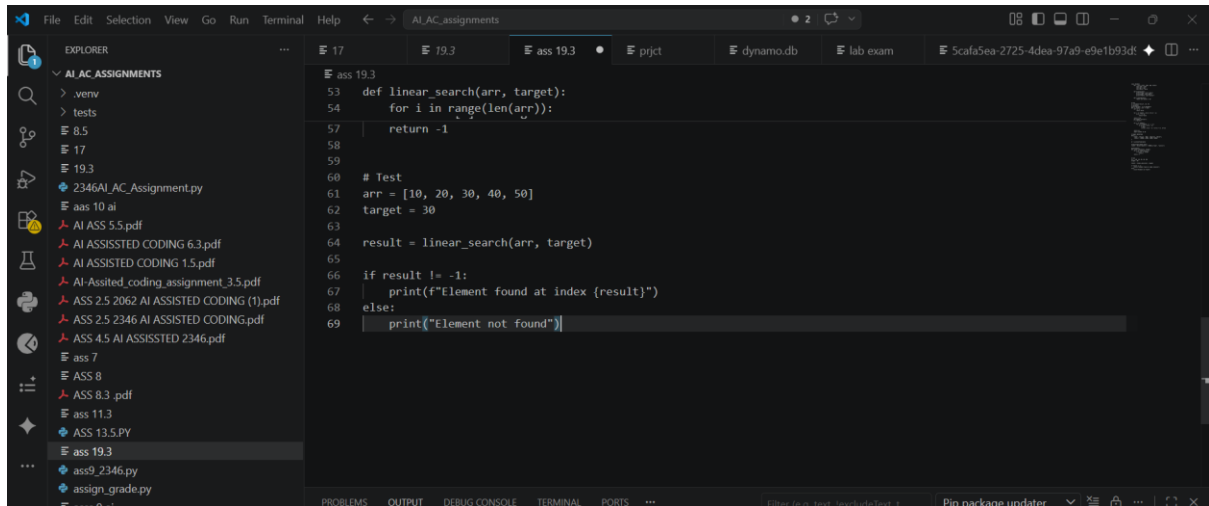
Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - o Syntax differences
 - o Library support

o Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.



```
File Edit Selection View Go Run Terminal Help
AI_AC_assignments
2
08 10 2023

EXPLORER
AI_AC_ASSIGNMENTS
  .venv
  tests
  8.5
  17
  19.3
  2346AI_AC_Assignment.py
  aas 10 ai
  AI ASS 5.5.pdf
  AI ASSISTED CODING 6.3.pdf
  AI ASSISTED CODING 1.5.pdf
  AI-Assited_coding_assignment_3.5.pdf
  ASS 2.5 2062 AI ASSISTED CODING (1).pdf
  ASS 2.5 2346 AI ASSISTED CODING.pdf
  ASS 4.5 AI ASSISTED 2346.pdf
  ass 7
  ASS 8
  ASS 8.3 .pdf
  ass 11.3
  ASS 13.5.PY
  ass 19.3
  ass9_2346.py
  assign_grade.py
  ass 8 ai

ass 19.3
53 def linear_search(arr, target):
54     for i in range(len(arr)):
57         return -1
58
59
60 # Test
61 arr = [10, 20, 30, 40, 50]
62 target = 30
63
64 result = linear_search(arr, target)
65
66 if result != -1:
67     print(f"Element found at index {result}")
68 else:
69     print("Element not found")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Pip package updater
```